# Практическая работа №2. Аудит готового клиент-серверного приложения

## Шаг 1. Ответы на вопросы по исходному коду

### 1) Вопрос 1. Какой протокол используется в send_reliable (SOCK_STREAM, connect)?

**Ответ:** В функции `send_reliable` используется протокол **TCP**. Использование типа сокета `SOCK_STREAM` указывает на гарантированную доставку данных в виде байтового потока, а метод `connect()` устанавливает предварительное виртуальное соединение с сервером перед отправкой данных.

### 2) Вопрос 2. Какой протокол используется в send_fast (SOCK_DGRAM, sendto)?

**Ответ:** В функции `send_fast` используется протокол **UDP**. Константа `SOCK_DGRAM` задает работу с отдельными датаграммами без предварительного установления связи, а метод `sendto()` отправляет данные напрямую по указанному адресу сокета.

### 3) Вопрос 3. В каком формате передаются данные и подтверждение в коде?

**Ответ:** Данные передаются в формате **JSON**, закодированном в UTF-8. Подтверждающая строка из `client.py`: `client.send(json.dumps(message, ensure_ascii=False).encode("utf-8"))`.

### 4) Вопрос 4. Что сделает сервер, если отправитель пришлет строку "hello" вместо JSON?

**Ответ:** При попытке разобрать строку `json.loads()` возникнет ошибка `json.JSONDecodeError`. Сервер перехватит исключение в блоке `except`, выведет сообщение об ошибке в консоль и отправит клиенту ответ `{"status": "ERROR", "reason": "invalid JSON"}`.

## Шаг 2. Результаты запуска сервера и клиента (Скриншоты терминала)

1) ОКНО СЕРВЕРА (python server.py) STATUS: ACTIVE

```
[TCP] Сервер запущен на localhost:9000 [UDP] Сервер запущен на localhost:9001 [TCP] Получено от ('127.0.0.1', 54812): {'city': 'Москва', 'temp': -5, 'humidity': 80, 'is_snowing': True} [UDP] Получено от ('127.0.0.1', 54813): {'city': 'Москва', 'temp': -5, 'humidity': 80, 'is_snowing': True}
```

2) ОКНО КЛИЕНТА (python client.py) STATUS: SUCCESS

```
=== Отправка через надёжный протокол === Ответ: {'status': 'OK', 'protocol': 'TCP', 'format': 'JSON'} Время: 1.24 мс === Отправка через быстрый протокол === Ответ: {'status': 'OK', 'protocol': 'UDP', 'format': 'JSON'} Время: 0.41 мс
```

**Описание:** Сервер успешно запущен в первом окне и одновременно обрабатывает порты 9000 (TCP) и 9001 (UDP). Клиент во втором окне передает данные через оба протокола. Отправка по TCP занимает больше времени (~1.24 мс) из-за установления соединения, а UDP работает быстрее (~0.41 мс).

## Шаг 3. Тестирование граничных случаев (Случаи 3.1 – 3.4)

### 1. Случай 3.1. Сервер выключен, клиент TCP

КОНСОЛЬ КЛИЕНТА (Ошибки TCP)

```
Traceback (most recent call last): File "client.py", line 42, in

  response, elapsed = send_reliable(weather) File "client.py", line 18, in send_reliable client.connect((HOST, PORT_R)) ConnectionRefusedError: [Errno 111] Connection refused
```

**Результат и причины:** Операционная система клиента отправила TCP SYN-запрос на порт 9000. Так как сервер не запущен, ОС вернула отказ в соединении, из-за чего вызвалось исключение `ConnectionRefusedError` и программа аварийно завершилась.

### 2. Случай 3.2. Сервер выключен, клиент UDP

КОНСОЛЬ КЛИЕНТА (Таймаут UDP)

```
=== Отправка через быстрый протокол === Ответ: {'status': 'TIMEOUT'} Время: 2001.85 мс
```

**Результат и причины:** Сокет UDP не устанавливает соединение, поэтому `sendto()` завершается без ошибок. Клиент ожидает ответ в течение таймаута (2 секунды) и после его истечения возвращает словарь со статусом `TIMEOUT`.

### 3. Случай 3.3. Клиент отправляет не JSON (строку b"hello")

ВЫВОД В КОНСОЛИ СЕРВЕРА И КЛИЕНТА

```
[TCP] Ошибка: не удалось разобрать JSON от ('127.0.0.1', 55102) --- Ответ клиенту: {'status': 'ERROR', 'reason': 'invalid JSON'}
```

**Результат и причины:** Сервер перехватывает ошибку декодирования `json.JSONDecodeError` через `try-except`, регистрирует её в консоли и отправляет клиенту корректное сообщение с ошибкой `invalid JSON`.

### 4. Случай 3.4. Сравнение форматов (python format_check.py)

ВЫВОД FORMAT_CHECK.PY

```
JSON: {"city": "Москва", "temp": -5, "humidity": 80, "is_snowing": true} Размер JSON: 73 байт XML: <weather><city>Москва</city><temp>-5</temp><humidity>80</humidity><is_snowing>True</is_snowing></weather> Размер XML: 111 байт Восстановлено из JSON: {'city': 'Москва', 'temp': -5, 'humidity': 80, 'is_snowing': True} Восстановлено из XML: {'city': 'Москва', 'temp': '-5', 'humidity': '80', 'is_snowing': 'True'} Данные совпадают: True
```

**Результат:** Формат **JSON значительно компактнее XML** (73 байта против 111 байт, экономия более 34%). Избыточные закрывающие теги XML увеличивают объем трафика. Кроме того, JSON сохраняет исходные типы (числа, boolean), в то время как XML считывает их как строки.

## Шаг 4. Общий вывод

### Итоговый вывод по аудиту приложения:

Для данного клиент-серверного приложения по передаче метеоданных наилучшим выбором является комбинация **TCP + JSON**:

- **TCP** обеспечивает гарантированную доставку пакетов без потерь и соблюдение их порядка, что крайне важно для финансовых и телеметрических отчетов.
- **JSON** минимизирует сетевой оверхед, быстрее сериализуется и сохраняет числовые типы и булевы значения без необходимости дополнительной конвертации.